

Complete Pandas & NumPy Cheat Sheet

NumPy Fundamentals

Importing and Basic Array Creation

```
python

import numpy as np

# Creating arrays
a = np.array([1, 2, 3])          # 1D array
b = np.array([[1, 2, 3], [4, 5, 6]])  # 2D array
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]]) # 3D array

# Special arrays
np.zeros((3, 4))                # 3x4 array of zeros
np.ones((2, 3, 4))              # 2x3x4 array of ones
np.empty((2, 2))                # Uninitialized array
np.full((3, 3), 7)              # 3x3 array filled with 7
np.eye(4)                       # 4x4 identity matrix
np.arange(0, 10, 2)              # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)            # 5 evenly spaced values between 0 and 1
np.random.rand(3, 4)            # Random values [0, 1) in 3x4 shape
np.random.randn(3, 4)           # Random normal distribution
np.random.randint(0, 10, (3, 4)) # Random integers
```

Array Attributes

```
python

arr = np.array([[1, 2, 3], [4, 5, 6]])

arr.shape    # (2, 3) - dimensions
arr.size     # 6 - total elements
arr.ndim     # 2 - number of dimensions
arr.dtype    # data type (int64, float64, etc.)
arr.itemsize # bytes per element
arr.nbytes   # total bytes consumed
```

Array Indexing and Slicing

```
python
```

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
```

```
# Basic indexing
```

```
arr[0]      # First row: [1, 2, 3, 4]
```

```
arr[1, 2]   # Element at row 1, col 2: 7
```

```
arr[-1]     # Last row: [9, 10, 11, 12]
```

```
# Slicing
```

```
arr[0:2]    # First two rows
```

```
arr[:, 1]   # All rows, second column: [2, 6, 10]
```

```
arr[1:, 2:] # Rows 1+, columns 2+
```

```
arr[::2, ::2] # Every other row and column
```

```
# Boolean indexing
```

```
arr[arr > 5] # Elements greater than 5
```

```
arr[(arr > 5) & (arr < 10)] # Multiple conditions
```

```
arr[arr % 2 == 0] # Even numbers
```

```
# Fancy indexing
```

```
arr[[0, 2]] # Rows 0 and 2
```

```
arr[[0, 1], [1, 2]] # Elements at (0,1) and (1,2)
```

Array Operations

```
python
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
# Arithmetic operations (element-wise)
```

```
a + b      # [5, 7, 9]
```

```
a - b      # [-3, -3, -3]
```

```
a * b      # [4, 10, 18]
```

```
a / b      # [0.25, 0.4, 0.5]
```

```
a ** 2     # [1, 4, 9]
```

```
a % 2      # [1, 0, 1]
```

```
# Universal functions
```

```
np.sqrt(a)  # Square root
```

```
np.exp(a)   # Exponential
```

```
np.log(a)   # Natural log
```

```
np.sin(a)   # Sine
```

```
np.cos(a)   # Cosine
```

```
np.abs(a)   # Absolute value
```

```
np.ceil(a)  # Ceiling
```

```
np.floor(a) # Floor
```

```
np.round(a, 2) # Round to 2 decimals
```

```
# Comparison operations
```

```
a > 2       # [False, False, True]
```

```
a == b      # Element-wise comparison
```

```
np.allclose(a, b) # Check if arrays are close (with tolerance)
```

Array Manipulation

```
python
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Reshaping
arr.reshape(3, 2)    # Reshape to 3x2
arr.flatten()        # Flatten to 1D
arr.ravel()          # Flatten (returns view if possible)
arr.T                # Transpose
arr.transpose()       # Transpose (explicit)

# Stacking and splitting
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
np.vstack([a, b])     # Stack vertically
np.hstack([a, b])     # Stack horizontally
np.concatenate([a, b]) # Concatenate
np.stack([a, b])      # Stack along new axis
np.split(arr, 2)      # Split into 2 equal arrays

# Adding/removing elements
np.append(a, [7, 8])  # Append elements
np.insert(a, 1, 99)   # Insert 99 at index 1
np.delete(a, 1)        # Delete element at index 1
np.unique(a)           # Unique elements
```

Aggregation Functions

```
python
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
# Basic aggregations
```

```
arr.sum()      # Sum of all elements
```

```
arr.mean()     # Mean
```

```
arr.std()      # Standard deviation
```

```
arr.var()      # Variance
```

```
arr.min()      # Minimum
```

```
arr.max()      # Maximum
```

```
arr.prod()     # Product of all elements
```

```
# Axis-specific operations
```

```
arr.sum(axis=0) # Sum along columns: [5, 7, 9]
```

```
arr.sum(axis=1) # Sum along rows: [6, 15]
```

```
arr.mean(axis=0) # Mean of each column
```

```
arr.cumsum()    # Cumulative sum
```

```
arr.cumprod()   # Cumulative product
```

```
# Position functions
```

```
arr.argmin()    # Index of minimum
```

```
arr.argmax()    # Index of maximum
```

```
np.where(arr > 3) # Indices where condition is true
```

```
np.percentile(arr, 50) # 50th percentile (median)
```

```
np.median(arr)   # Median
```

```
np.quantile(arr, 0.75) # 75th quantile
```

Linear Algebra

```
python
```

```
a = np.array([[1, 2], [3, 4]])
```

```
b = np.array([[5, 6], [7, 8]])
```

```
# Matrix operations
```

```
np.dot(a, b)    # Matrix multiplication
```

```
a @ b           # Matrix multiplication (Python 3.5+)
```

```
np.matmul(a, b) # Matrix multiplication
```

```
np.linalg.inv(a) # Matrix inverse
```

```
np.linalg.det(a) # Determinant
```

```
np.linalg.eig(a) # Eigenvalues and eigenvectors
```

```
np.linalg.solve(a, b) # Solve linear system
```

```
np.trace(a)      # Trace (sum of diagonal)
```

```
np.diag(a)       # Extract diagonal
```

Broadcasting

```
python

# Broadcasting allows operations on arrays of different shapes
a = np.array([[1, 2, 3], [4, 5, 6]]) # (2, 3)
b = np.array([10, 20, 30])          # (3,)
result = a + b # b is broadcast to (2, 3): [[11, 22, 33], [14, 25, 36]]

# More examples
a = np.array([[1], [2], [3]]) # (3, 1)
b = np.array([10, 20, 30])    # (3,)
result = a + b # Result is (3, 3)
```

Pandas Fundamentals

Importing and Basic Structures

```
python

import pandas as pd

# Series - 1D labeled array
s = pd.Series([1, 2, 3, 4])
s = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s = pd.Series({'a': 1, 'b': 2, 'c': 3})

# DataFrame - 2D labeled data structure
df = pd.DataFrame({
    'col1': [1, 2, 3],
    'col2': ['a', 'b', 'c'],
    'col3': [1.5, 2.5, 3.5]
})

# From various sources
df = pd.DataFrame(np.random.randn(4, 3), columns=['A', 'B', 'C'])
df = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])
```

Reading and Writing Data

```
python
```

```
# CSV files
df = pd.read_csv('file.csv')
df = pd.read_csv('file.csv', sep=';', encoding='utf-8')
df = pd.read_csv('file.csv', index_col=0, parse_dates=['date_col'])
df.to_csv('output.csv', index=False)

# Excel files
df = pd.read_excel('file.xlsx', sheet_name='Sheet1')
df.to_excel('output.xlsx', sheet_name='Data', index=False)

# JSON files
df = pd.read_json('file.json')
df.to_json('output.json', orient='records')

# SQL databases
df = pd.read_sql('SELECT * FROM table', connection)
df.to_sql('table_name', connection, if_exists='replace')

# Other formats
df = pd.read_parquet('file.parquet')
df = pd.read_html('url_or_html_string')[0] # Returns list of DataFrames
df.to_pickle('file.pkl')
df = pd.read_pickle('file.pkl')
```

DataFrame Inspection

```
python

df.head(10)      # First 10 rows (default 5)
df.tail(10)      # Last 10 rows
df.sample(5)     # Random 5 rows
df.shape         # (rows, columns)
df.info()        # Column types, non-null counts, memory usage
df.describe()    # Statistical summary
df.dtypes        # Data types of each column
df.columns       # Column names
df.index         # Index (row labels)
df.values        # NumPy array of values
df.nunique()     # Number of unique values per column
df.memory_usage() # Memory usage per column
```

Indexing and Selection

```
python
```

Column selection

`df['col1']` *# Single column (Series)*

`df[['col1', 'col2']]` *# Multiple columns (DataFrame)*

Row selection

`df[0:3]` *# Rows 0-2 by position*

`df['2020-01-01':'2020-12-31']` *# Slice by index labels*

loc - label-based indexing

`df.loc[0]` *# Row with label 0*

`df.loc[0:3]` *# Rows 0 to 3 (inclusive)*

`df.loc[:, 'col1']` *# All rows, column 'col1'*

`df.loc[0:3, ['col1', 'col2']]` *# Rows 0-3, specific columns*

`df.loc[df['col1'] > 5]` *# Boolean indexing*

iloc - position-based indexing

`df.iloc[0]` *# First row*

`df.iloc[0:3]` *# Rows 0-2*

`df.iloc[:, 0]` *# All rows, first column*

`df.iloc[0:3, 0:2]` *# Rows 0-2, columns 0-1*

`df.iloc[[0, 2, 4]]` *# Specific rows by position*

at and iat - fast scalar access

`df.at[0, 'col1']` *# Value at row 0, column 'col1'*

`df.iat[0, 0]` *# Value at row 0, column 0 (by position)*

Boolean indexing

`df[df['col1'] > 5]`

`df[(df['col1'] > 5) & (df['col2'] < 10)]`

`df[df['col1'].isin([1, 2, 3])]`

`df[df['col1'].between(5, 10)]`

`df[~df['col1'].isin([1, 2, 3])]` *# NOT in list*

Data Cleaning

python

Handling missing data

```
df.isnull()          # Boolean mask of null values
df.notnull()         # Boolean mask of non-null values
df.isnull().sum()    # Count nulls per column
df.dropna()          # Drop rows with any null
df.dropna(axis=1)     # Drop columns with any null
df.dropna(how='all')  # Drop rows where all values are null
df.dropna(thresh=2)   # Keep rows with at least 2 non-null values
df.fillna(0)          # Fill nulls with 0
df.fillna(method='ffill') # Forward fill
df.fillna(method='bfill') # Backward fill
df.fillna(df.mean())  # Fill with column mean
df.interpolate()      # Interpolate missing values
```

Removing duplicates

```
df.duplicated()       # Boolean mask of duplicate rows
df.drop_duplicates()   # Remove duplicate rows
df.drop_duplicates(subset=['col1']) # Based on specific columns
df.drop_duplicates(keep='last') # Keep last occurrence
```

Replacing values

```
df.replace(0, np.nan) # Replace 0 with NaN
df.replace({'col1': {0: 100}}) # Replace in specific column
df.replace([0, 1], [100, 200]) # Multiple replacements
df['col1'].replace({0: 100, 1: 200}) # Dictionary mapping
```

Renaming

```
df.rename(columns={'old': 'new'})
df.rename(index={0: 'first'})
df.columns = ['new1', 'new2', 'new3']
df.rename(str.lower, axis='columns') # Lowercase all columns
```

Data types

```
df['col1'].astype(int)
df['col1'].astype('category')
pd.to_numeric(df['col1'], errors='coerce') # Convert to numeric, invalid -> NaN
pd.to_datetime(df['date_col'])
```

Data Manipulation

python

Adding/removing columns

```
df['new_col'] = df['col1'] + df['col2']
df['new_col'] = df.apply(lambda row: row['col1'] * 2, axis=1)
df.insert(1, 'new_col', [1, 2, 3]) # Insert at position 1
df.drop('col1', axis=1) # Drop column
df.drop(['col1', 'col2'], axis=1) # Drop multiple columns
df.pop('col1') # Remove and return column
```

Adding/removing rows

```
df.loc[len(df)] = [1, 2, 3] # Add row at end
df.drop(0, axis=0) # Drop row by label
df.drop([0, 1, 2]) # Drop multiple rows
```

Sorting

```
df.sort_values('col1') # Sort by column
df.sort_values(['col1', 'col2'], ascending=[True, False])
df.sort_index() # Sort by index
df.nlargest(5, 'col1') # Top 5 values
df.nsmallest(5, 'col1') # Bottom 5 values
```

Filtering

```
df.query('col1 > 5')
df.query('col1 > 5 and col2 < 10')
df.query('col1 in [1, 2, 3]')
```

Apply functions

```
df['col1'].apply(lambda x: x * 2)
df.apply(lambda x: x.max() - x.min()) # Apply to each column
df.applymap(lambda x: x * 2) # Apply to every element (deprecated, use map)
df.map(lambda x: x * 2) # Apply to every element
```

String operations

```
df['col1'].str.lower()
df['col1'].str.upper()
df['col1'].str.strip()
df['col1'].str.replace('old', 'new')
df['col1'].str.contains('pattern')
df['col1'].str.split(',')
df['col1'].str.extract(r'(\d+)') # Extract with regex
df['col1'].str.len()
df['col1'].str.startswith('A')
df['col1'].str.endswith('Z')
```

GroupBy Operations

Basic grouping

```
grouped = df.groupby('col1')
grouped.size()      # Count of each group
grouped.groups      # Dictionary of groups
grouped.get_group('A') # Get specific group
```

Aggregations

```
df.groupby('col1').sum()
df.groupby('col1').mean()
df.groupby('col1').agg(['sum', 'mean', 'count'])
df.groupby('col1')['col2'].sum() # Aggregate specific column
df.groupby(['col1', 'col2']).sum() # Multiple groupby columns
```

Custom aggregations

```
df.groupby('col1').agg({
    'col2': 'sum',
    'col3': 'mean',
    'col4': ['min', 'max']
})
```

Transform and filter

```
df.groupby('col1')['col2'].transform('sum') # Broadcast result back
df.groupby('col1').filter(lambda x: len(x) > 3) # Keep groups with >3 rows
```

Iterating groups

```
for name, group in df.groupby('col1'):
    print(name, group)
```

Merging and Joining

python

Concatenation

```
pd.concat([df1, df2])      # Vertical stack
pd.concat([df1, df2], axis=1) # Horizontal stack
pd.concat([df1, df2], ignore_index=True) # Reset index
```

Merging (SQL-style joins)

```
pd.merge(df1, df2, on='key')      # Inner join
pd.merge(df1, df2, on='key', how='left') # Left join
pd.merge(df1, df2, on='key', how='right') # Right join
pd.merge(df1, df2, on='key', how='outer') # Outer join
pd.merge(df1, df2, left_on='key1', right_on='key2') # Different key names
pd.merge(df1, df2, left_index=True, right_index=True) # Join on index
```

Join (index-based merge)

```
df1.join(df2)
df1.join(df2, how='left', lsuffix='_left', rsuffix='_right')
```

Append (deprecated, use concat)

```
pd.concat([df1, df2], ignore_index=True)
```

Pivot Tables and Reshaping

python

Pivot table

```
df.pivot(index='row_col', columns='col_col', values='value_col')
df.pivot_table(values='value_col', index='row_col',
               columns='col_col', aggfunc='mean')
df.pivot_table(values='value_col', index='row_col',
               columns='col_col', aggfunc=['sum', 'mean'])
```

Melt (unpivot)

```
pd.melt(df, id_vars=['id'], value_vars=['col1', 'col2'])
pd.melt(df, id_vars=['id'], var_name='variable', value_name='value')
```

Stack and unstack

```
df.stack()      # Pivot columns to rows
df.unstack()     # Pivot rows to columns
df.unstack(level=0) # Unstack specific level
```

Crosstab

```
pd.crosstab(df['col1'], df['col2'])
pd.crosstab(df['col1'], df['col2'], normalize='all') # Show percentages
```

Time Series

```
python

# Creating datetime index
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)
date_range = pd.date_range('2020-01-01', '2020-12-31', freq='D')

# Accessing datetime components
df['date'].dt.year
df['date'].dt.month
df['date'].dt.day
df['date'].dt.dayofweek
df['date'].dt.hour
df['date'].dt.date
df['date'].dt.time

# Resampling
df.resample('M').sum() # Monthly sum
df.resample('W').mean() # Weekly mean
df.resample('Q').last() # Quarterly last value
df.resample('2H').mean() # Every 2 hours

# Rolling windows
df.rolling(window=7).mean() # 7-day moving average
df.rolling(window=7).sum()
df.rolling(window=7, min_periods=1).mean() # Handle edge cases
df.expanding().mean() # Expanding window

# Shifting
df.shift(1) # Shift down by 1
df.shift(-1) # Shift up by 1
df.diff() # Difference from previous row
df.pct_change() # Percentage change

# Time zone operations
df.tz_localize('UTC')
df.tz_convert('US/Eastern')
```

Advanced Operations

```
python
```

```

# Multi-indexing
df.set_index(['col1', 'col2'])
df.reset_index()
df.xs('key', level='level_name') # Cross-section

# Categorical data
df['col'] = df['col'].astype('category')
df['col'].cat.categories
df['col'].cat.codes
df['col'].cat.add_categories(['new'])
df['col'].cat.remove_categories(['old'])

# Window functions
df['rank'] = df['col1'].rank()
df['cumsum'] = df['col1'].cumsum()
df['cummax'] = df['col1'].cummax()
df['cummin'] = df['col1'].cummin()

# Binning
pd.cut(df['col1'], bins=5) # Equal-width bins
pd.qcut(df['col1'], q=4) # Equal-frequency bins (quartiles)
pd.cut(df['col1'], bins=[0, 10, 20, 30], labels=['low', 'med', 'high'])

# One-hot encoding
pd.get_dummies(df['col1'])
pd.get_dummies(df, columns=['col1', 'col2'])

# Correlation and covariance
df.corr() # Correlation matrix
df.cov() # Covariance matrix
df['col1'].corr(df['col2']) # Correlation between two columns

# Value counts
df['col1'].value_counts()
df['col1'].value_counts(normalize=True) # Percentages
df['col1'].value_counts(bins=5)

# Sample data
df.sample(n=10) # 10 random rows
df.sample(frac=0.1) # 10% random sample
df.sample(n=10, replace=True) # With replacement

```

Performance Optimization

python

```
# Vectorized operations (avoid loops)
df['result'] = df['col1'] * df['col2'] # Fast
# Avoid: df['result'] = df.apply(lambda x: x['col1'] * x['col2'], axis=1)

# Use categorical for repeated strings
df['col'] = df['col'].astype('category')

# Read chunks for large files
for chunk in pd.read_csv('large_file.csv', chunksize=10000):
    process(chunk)

# Use efficient data types
df['int_col'] = df['int_col'].astype('int32') # Instead of int64
df['float_col'] = df['float_col'].astype('float32')

# Query instead of boolean indexing for large DataFrames
df.query('col1 > 5') # Faster than df[df['col1'] > 5]

# Use eval for complex expressions
df.eval('result = col1 + col2 * col3')

# Avoid chained indexing
df.loc[df['col1'] > 5, 'col2'] = 0 # Good
# Avoid: df[df['col1'] > 5]['col2'] = 0 # SettingWithCopyWarning
```

Common Real-World Patterns

Data Cleaning Pipeline

```
python
```

```
# Complete cleaning workflow
```

```
df = pd.read_csv('data.csv')
```

```
# 1. Handle missing values
```

```
df = df.dropna(thresh=len(df.columns)*0.5) # Drop rows with >50% missing
```

```
df = df.fillna(df.mean(numeric_only=True)) # Fill numeric with mean
```

```
# 2. Remove duplicates
```

```
df = df.drop_duplicates()
```

```
# 3. Fix data types
```

```
df['date'] = pd.to_datetime(df['date'])
```

```
df['category'] = df['category'].astype('category')
```

```
# 4. Handle outliers
```

```
Q1 = df['numeric_col'].quantile(0.25)
```

```
Q3 = df['numeric_col'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
df = df[~((df['numeric_col'] < (Q1 - 1.5 * IQR)) |  
          (df['numeric_col'] > (Q3 + 1.5 * IQR)))]
```

```
# 5. Standardize text
```

```
df['text_col'] = df['text_col'].str.lower().str.strip()
```

Data Aggregation and Reporting

```
python
```

```
# Create summary report
```

```
summary = df.groupby('category').agg({  
    'sales': ['sum', 'mean', 'count'],  
    'profit': 'sum',  
    'customer_id': 'nunique'  
}).round(2)
```

```
# Rename columns
```

```
summary.columns = ['total_sales', 'avg_sales', 'num_transactions',  
                   'total_profit', 'unique_customers']
```

```
# Add percentage columns
```

```
summary['pct_of_total'] = summary['total_sales'] / summary['total_sales'].sum() * 100
```

Time Series Analysis

```
python
```



```
# Prepare time series data
df['date'] = pd.to_datetime(df['date'])
df = df.set_index('date').sort_index()

# Calculate metrics
df['7day_ma'] = df['value'].rolling(7).mean()
df['pct_change'] = df['value'].pct_change()
df['ytd'] = df['value'].cumsum()

# Resample to monthly
monthly = df.resample('M').agg({
    'value': 'sum',
    'transactions': 'count',
    'customer_id': 'nunique'
})
```

Data Transformation for ML

```
python

# Prepare data for machine learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Handle categorical variables
df_encoded = pd.get_dummies(df, columns=['category', 'region'])

# Separate features and target
X = df_encoded.drop('target', axis=1)
y = df_encoded['target']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Quick Reference

NumPy vs Pandas

Operation	NumPy	Pandas
Create	<code>np.array([1,2,3])</code>	<code>pd.Series([1,2,3])</code>
Indexing	<code>arr[0]</code>	<code>df.iloc[0]</code> or <code>df.loc[0]</code>
Slicing	<code>arr[1:3]</code>	<code>df[1:3]</code> or <code>df.iloc[1:3]</code>
Boolean	<code>arr[arr > 5]</code>	<code>df[df['col'] > 5]</code>
Aggregation	<code>arr.sum()</code>	<code>df.sum()</code> or <code>df['col'].sum()</code>

Common Gotchas

```
python

# 1. Chained indexing - causes SettingWithCopyWarning
df[df['A'] > 0]['B'] = 1 # Bad
df.loc[df['A'] > 0, 'B'] = 1 # Good

# 2. Modifying during iteration
for idx, row in df.iterrows():
    df.at[idx, 'new'] = row['old'] * 2 # Slow but works
# Better: df['new'] = df['old'] * 2

# 3. Comparing arrays
(arr1 == arr2).all() # Check if all elements equal
np.array_equal(arr1, arr2) # Better way

# 4. Integer division
np.array([1, 2, 3]) / np.array([2, 2, 2]) # Returns float
np.array([1, 2, 3]) // np.array([2, 2, 2]) # Integer division
```

This cheat sheet covers the essential operations you'll use in real-world data analysis and manipulation tasks.